

BryceEmery-Assignment-2

May 20, 2026

1 Assignment #2 - Recursive Feature Elimination with Linear Regression

Bryce Emery

This code uses the provided RFE example code as a reference source.

```
[16]: # Initialization
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# ref: https://scikit-learn.org/stable/datasets/toy_dataset.
#      ↪html#diabetes-dataset
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFE
from sklearn.model_selection import cross_val_score, train_test_split

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

np.random.seed(42)
# ref: https://www.geeksforgeeks.org/python/
#      ↪formatting-integer-column-of-dataframe-in-pandas/
# I don't want scientific notation for the describe functions
pd.options.display.float_format = '{: .8f}'.format
```

1.1 Task 1 - Data set Exploration

```
[17]: # load dataset
diabetes_data = load_diabetes()

X = pd.DataFrame(data=diabetes_data.data,
                 columns=diabetes_data.feature_names)
y = diabetes_data.target

# standardize features
```

```
#scalar = StandardScaler()
#X_scaled = scalar.fit_transform(X)
```

```
[18]: print(X.head)
print(f"Target: {y}")
```

```
<bound method NDFrame.head of
bp          s1          s2  \
0    0.03807591  0.05068012  0.06169621  0.02187239 -0.04422350 -0.03482076
1   -0.00188202 -0.04464164 -0.05147406 -0.02632753 -0.00844872 -0.01916334
2    0.08529891  0.05068012  0.04445121 -0.00567042 -0.04559945 -0.03419447
3   -0.08906294 -0.04464164 -0.01159501 -0.03665608  0.01219057  0.02499059
4    0.00538306 -0.04464164 -0.03638469  0.02187239  0.00393485  0.01559614
..          ...          ...          ...          ...          ...          ...
437  0.04170844  0.05068012  0.01966154  0.05974375 -0.00569682 -0.00256647
438 -0.00551455  0.05068012 -0.01590626 -0.06764174  0.04934130  0.07916528
439  0.04170844  0.05068012 -0.01590626  0.01729339 -0.03734373 -0.01383982
440 -0.04547248 -0.04464164  0.03906215  0.00121528  0.01631843  0.01528299
441 -0.04547248 -0.04464164 -0.07303030 -0.08141314  0.08374012  0.02780893
```

```
          s3          s4          s5          s6
0   -0.04340085 -0.00259226  0.01990749 -0.01764613
1    0.07441156 -0.03949338 -0.06833155 -0.09220405
2   -0.03235593 -0.00259226  0.00286131 -0.02593034
3   -0.03603757  0.03430886  0.02268774 -0.00936191
4    0.00814208 -0.00259226 -0.03198764 -0.04664087
..          ...          ...          ...          ...
437 -0.02867429 -0.00259226  0.03119260  0.00720652
438 -0.02867429  0.03430886 -0.01811369  0.04448548
439 -0.02499266 -0.01107952 -0.04688253  0.01549073
440 -0.02867429  0.02655962  0.04452873 -0.02593034
441  0.17381578 -0.03949338 -0.00422151  0.00306441
```

```
[442 rows x 10 columns]>
```

```
Target: [151.  75. 141. 206. 135.  97. 138.  63. 110. 310. 101.  69. 179. 185.
 118. 171. 166. 144.  97. 168.  68.  49.  68. 245. 184. 202. 137.  85.
 131. 283. 129.  59. 341.  87.  65. 102. 265. 276. 252.  90. 100.  55.
  61.  92. 259.  53. 190. 142.  75. 142. 155. 225.  59. 104. 182. 128.
  52.  37. 170. 170.  61. 144.  52. 128.  71. 163. 150.  97. 160. 178.
  48. 270. 202. 111.  85.  42. 170. 200. 252. 113. 143.  51.  52. 210.
  65. 141.  55. 134.  42. 111.  98. 164.  48.  96.  90. 162. 150. 279.
  92.  83. 128. 102. 302. 198.  95.  53. 134. 144. 232.  81. 104.  59.
246. 297. 258. 229. 275. 281. 179. 200. 200. 173. 180.  84. 121. 161.
  99. 109. 115. 268. 274. 158. 107.  83. 103. 272.  85. 280. 336. 281.
118. 317. 235.  60. 174. 259. 178. 128.  96. 126. 288.  88. 292.  71.
197. 186.  25.  84.  96. 195.  53. 217. 172. 131. 214.  59.  70. 220.
268. 152.  47.  74. 295. 101. 151. 127. 237. 225.  81. 151. 107.  64.
138. 185. 265. 101. 137. 143. 141.  79. 292. 178.  91. 116.  86. 122.]
```

```

72. 129. 142. 90. 158. 39. 196. 222. 277. 99. 196. 202. 155. 77.
191. 70. 73. 49. 65. 263. 248. 296. 214. 185. 78. 93. 252. 150.
77. 208. 77. 108. 160. 53. 220. 154. 259. 90. 246. 124. 67. 72.
257. 262. 275. 177. 71. 47. 187. 125. 78. 51. 258. 215. 303. 243.
91. 150. 310. 153. 346. 63. 89. 50. 39. 103. 308. 116. 145. 74.
45. 115. 264. 87. 202. 127. 182. 241. 66. 94. 283. 64. 102. 200.
265. 94. 230. 181. 156. 233. 60. 219. 80. 68. 332. 248. 84. 200.
55. 85. 89. 31. 129. 83. 275. 65. 198. 236. 253. 124. 44. 172.
114. 142. 109. 180. 144. 163. 147. 97. 220. 190. 109. 191. 122. 230.
242. 248. 249. 192. 131. 237. 78. 135. 244. 199. 270. 164. 72. 96.
306. 91. 214. 95. 216. 263. 178. 113. 200. 139. 139. 88. 148. 88.
243. 71. 77. 109. 272. 60. 54. 221. 90. 311. 281. 182. 321. 58.
262. 206. 233. 242. 123. 167. 63. 197. 71. 168. 140. 217. 121. 235.
245. 40. 52. 104. 132. 88. 69. 219. 72. 201. 110. 51. 277. 63.
118. 69. 273. 258. 43. 198. 242. 232. 175. 93. 168. 275. 293. 281.
72. 140. 189. 181. 209. 136. 261. 113. 131. 174. 257. 55. 84. 42.
146. 212. 233. 91. 111. 152. 120. 67. 310. 94. 183. 66. 173. 72.
49. 64. 48. 178. 104. 132. 220. 57.]

```

All features are normalized, with a mean of 0.0 and standard deviation of 0.0476.

1.1.1 Per Feature Descriptions:

age

```
[19]: print(f"Median: {X['age'].median()}")
      X[['age']].describe()
```

Median: 0.005383060374248237

```
[19]:
      age
count 442.00000000
mean   -0.00000000
std     0.04761905
min    -0.10722563
25%    -0.03729927
50%     0.00538306
75%     0.03807591
max     0.11072668

```

The age variable has a median of 0.00538, with minimum of -0.1072, and a maximum of 0.1107.

sex

```
[20]: print(f"Median: {X['sex'].median()}")
      X[['sex']].describe()
```

Median: -0.044641636506989144

```
[20]:
      sex
count 442.00000000
mean    0.00000000

```

```
std      0.04761905
min      -0.04464164
25%      -0.04464164
50%      -0.04464164
75%      0.05068012
max       0.05068012
```

The age variable has a median of -0.0446, with minimum of -0.0446, and a maximum of 0.0507.

Noticeably the data seems to exist in two states, at -0.0446 and 0.0507. This seems expectable as, this data was likely stored as a binary factors ('Male' or 'Female') before being normalized.

bmi

```
[21]: print(f"Median: {X['bmi'].median()}")
      X[['bmi']].describe()
```

```
Median: -0.007283766209687899
```

```
[21]:          bmi
count  442.00000000
mean    -0.00000000
std      0.04761905
min     -0.09027530
25%     -0.03422907
50%     -0.00728377
75%      0.03124802
max      0.17055523
```

The bmi (body mass index) variable has a median of -0.00728, with minimum of -0.0903, and a maximum of 0.1706.

bp

```
[22]: print(f"Median: {X['bp'].median()}")
      X[['bp']].describe()
```

```
Median: -0.00567042229275739
```

```
[22]:          bp
count  442.00000000
mean    -0.00000000
std      0.04761905
min     -0.11239880
25%     -0.03665608
50%     -0.00567042
75%      0.03564379
max      0.13204362
```

The bp (blood pressure) variable has a median of -0.00567, with minimum of -0.1124, and a maximum of 0.1320.

s1-s2 - quantitative blood sample derived variables

```
[23]: print(f"Median: {X['s1'].median()}")
      print(X[['s1']].describe())
      print(f"Median: {X['s2'].median()}")
      print(X[['s2']].describe())
      print(f"Median: {X['s3'].median()}")
      print(X[['s3']].describe())
      print(f"Median: {X['s4'].median()}")
      print(X[['s4']].describe())
      print(f"Median: {X['s5'].median()}")
      print(X[['s5']].describe())
      print(f"Median: {X['s6'].median()}")
      print(X[['s6']].describe())
```

Median: -0.004320865536613489

s1

```
count 442.00000000
mean   -0.00000000
std     0.04761905
min    -0.12678067
25%    -0.03424784
50%    -0.00432087
75%     0.02835801
max     0.15391371
```

Median: -0.0038190651205350003

s2

```
count 442.00000000
mean    0.00000000
std     0.04761905
min    -0.11561307
25%    -0.03035840
50%    -0.00381907
75%     0.02984439
max     0.19878799
```

Median: -0.006584467611155497

s3

```
count 442.00000000
mean   -0.00000000
std     0.04761905
min    -0.10230705
25%    -0.03511716
50%    -0.00658447
75%     0.02931150
max     0.18117906
```

Median: -0.002592261998183278

s4

```
count 442.00000000
mean   -0.00000000
```

```

std      0.04761905
min      -0.07639450
25%      -0.03949338
50%      -0.00259226
75%      0.03430886
max      0.18523444
Median: -0.0019471710869220743

          s5
count 442.00000000
mean   0.00000000
std    0.04761905
min    -0.12609712
25%    -0.03324559
50%    -0.00194717
75%    0.03243232
max    0.13359728
Median: -0.0010776975004659671

          s6
count 442.00000000
mean   0.00000000
std    0.04761905
min    -0.13776723
25%    -0.03317903
50%    -0.00107770
75%    0.02791705
max    0.13561183

```

The s1 variable has a median of -0.0043, with minimum of -0.1238, and a maximum of 0.1539.

The s2 variable has a median of -0.0038, with minimum of -0.1156, and a maximum of 0.1988.

The s3 variable has a median of -0.0066, with minimum of -0.1023, and a maximum of 0.1812.

The s4 variable has a median of -0.0026, with minimum of -0.0764, and a maximum of 0.1852.

The s5 variable has a median of -0.0019, with minimum of -0.1261, and a maximum of 0.1336.

The s6 variable has a median of -0.0011, with minimum of -0.1378, and a maximum of 0.1356.

1.1.2 Training Testing Split

```
[24]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

1.2 Task 2 - Linear Regression Model

```
[25]: # Initialize Linear Regression Model
base_model = LinearRegression()
base_model.fit(X_train, y_train)
base_pred = base_model.predict(X_test)
```

```
base_r2 = r2_score(y_test, base_pred)
print(f"Base Model R2: {base_r2}")
```

Base Model R²: 0.4526027629719195

1.3 Task 3 - Implement Recursive Feature Elimination (RFE)

```
[26]: # Define recursive Feature Elimination
def perform_rfe_iteration(n_features_to_select, remaining_features=0):

    #
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    #   initial_features =
    model=LinearRegression()
    rfe = RFE(estimator=model, n_features_to_select=n_features_to_select)
    rfe.fit(X_train_scaled, y_train)

    # Get selected features
    selected_features = X.columns[rfe.support_]
    eliminated_features= X.columns[~rfe.support_]

    # Fit linear regression on selected features
    X_train_selected = X_train_scaled[:, rfe.support_]
    X_test_selected = X_test_scaled[:, rfe.support_]

    selected_model = LinearRegression()
    selected_model.fit(X_train_selected, y_train)
    selected_pred = selected_model.predict(X_test_selected)

    selected_r2 = r2_score(y_test, selected_pred)

    return selected_r2, selected_features, selected_model.coef_

def visualize_rfe_iterations(max_num_features=0, output=True, r_sqr_thresh=0.
    ↪01):

    coef_values = []
    rsqr_scores = []
    high_rsqr = 0
    num_features = []
```

```

rank_history = []
optimal_features_names = []

optimal_features_num = len(X_train.columns) if max_num_features == 0 else
↪max_num_features

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr = LinearRegression()
lr.fit(X_train, y_train)

# Initial feature coefficients
if (output):
    print("Initial Feature Coeffecients")
    for feature, coef in zip(diabetes_data.feature_names, lr.coef_):
        print(f"{feature}: {coef:.4f}")

for i in range(optimal_features_num, 0, -1):

    rsqr_score, sel_feats, sel_coefs = perform_rfe_iteration(i);
    if (output):
        print(f'--- RFE with {i} Features ---')

        print(f'Eliminated Features:')
        for feat in X.columns:
            if feat not in sel_feats:
                print(feat)
        print(f'\nSelected Features and Coefficients:')
        for feat, coef in zip(sel_feats, sel_coefs):
            print(f'{feat}: {coef:.4f}')
        print(f'\nR-Squared: {rsqr_score:.4f}\n')

    rsqr_scores.append(rsqr_score)
    rank_history.append(sel_feats)
    coef_values.append(sel_coefs)
    num_features.append(i)
    if (rsqr_score > r_sqr_thresh and rsqr_score > high_rsqr):
        high_rsqr=rsqr_score
        optimal_features_names = sel_feats
        optimal_features_num = i

# Plot full model
if (output):
    plt.figure(figsize=(12,6))

```



```

plt.plot(num_features, rsqr_scores, marker='o', label='R^2')
plt.axvline(optimal_features_num, color='red', linestyle='--',
↪label=f'Optimal Features ({optimal_features_num})')
plt.title("R^2 vs Number of Selected Features")
plt.xlabel("Number of Features")
plt.ylabel("R^2")
plt.legend()
plt.grid()
plt.show()
return rsqr_scores, rank_history, coef_values, num_features

#perform_rfe_iteration(1)

```

```
[27]: visualize_rfe_iterations()
```

Initial Feature Coeffecients

age: 37.9040
sex: -241.9644
bmi: 542.4288
bp: 347.7038
s1: -931.4888
s2: 518.0623
s3: 163.4200
s4: 275.3179
s5: 736.1989
s6: 48.6707

--- RFE with 10 Features ---

Eliminated Features:

Selected Features and Coefficients:

age: 1.7538
sex: -11.5118
bmi: 25.6071
bp: 16.8289
s1: -44.4489
s2: 24.6410
s3: 7.6770
s4: 13.1388
s5: 35.1612
s6: 2.3514

R-Squared: 0.4526

--- RFE with 9 Features ---

Eliminated Features:

age

Selected Features and Coefficients:

sex: -11.2590
bmi: 25.6246
bp: 17.1438
s1: -44.6808
s2: 25.1515
s3: 7.8828
s4: 12.9039
s5: 35.5552
s6: 2.5775

R-Squared: 0.4587

--- RFE with 8 Features ---

Eliminated Features:

age
s6

Selected Features and Coefficients:

sex: -11.1212
bmi: 25.9997
bp: 17.6075
s1: -45.2283
s2: 25.7598
s3: 8.0918
s4: 13.2544
s5: 36.3897

R-Squared: 0.4559

--- RFE with 7 Features ---

Eliminated Features:

age
s3
s6

Selected Features and Coefficients:

sex: -11.1978
bmi: 26.0527
bp: 17.5380
s1: -31.5246
s2: 16.3309
s4: 8.8353
s5: 31.7499

R-Squared: 0.4583

--- RFE with 6 Features ---

Eliminated Features:

age
s3
s4
s6

Selected Features and Coefficients:

sex: -10.2417
bmi: 26.3098
bp: 16.9487
s1: -40.6327
s2: 28.1146
s5: 38.3574

R-Squared: 0.4628

--- RFE with 5 Features ---

Eliminated Features:

age
sex
s3
s4
s6

Selected Features and Coefficients:

bmi: 28.2255
bp: 14.8418
s1: -31.2821
s2: 19.4831
s5: 34.8004

R-Squared: 0.4382

--- RFE with 4 Features ---

Eliminated Features:

age
sex
bp
s3
s4
s6

Selected Features and Coefficients:

bmi: 32.6426
s1: -28.2958
s2: 17.2633
s5: 37.4045

R-Squared: 0.4464

--- RFE with 3 Features ---

Eliminated Features:

age
sex
bp
s2
s3
s4
s6

Selected Features and Coefficients:

bmi: 34.8249
s1: -10.8959
s5: 32.4878

R-Squared: 0.4451

--- RFE with 2 Features ---

Eliminated Features:

age
sex
bp
s1
s2
s3
s4
s6

Selected Features and Coefficients:

bmi: 34.5616
s5: 26.8522

R-Squared: 0.4523

--- RFE with 1 Features ---

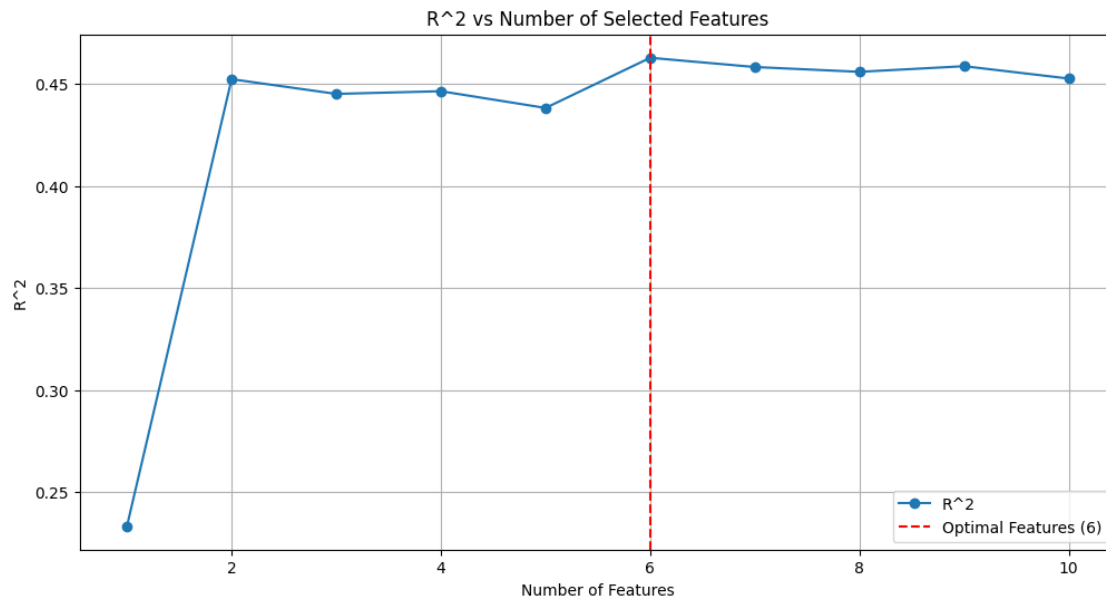
Eliminated Features:

age
sex
bp
s1
s2
s3
s4
s5
s6

Selected Features and Coefficients:

bmi: 47.1411

R-Squared: 0.2334



```
[27]: ([0.45260276297191926,
        0.4586588697581955,
        0.4559010325046122,
        0.45825530847242857,
        0.4627767079320301,
        0.4382013933899772,
        0.4464044355210377,
        0.4450948811323354,
        0.4522925957397932,
        0.2333503981587215],
        [Index(['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'],
              dtype='str'),
         Index(['sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'], dtype='str'),
         Index(['sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5'], dtype='str'),
         Index(['sex', 'bmi', 'bp', 's1', 's2', 's4', 's5'], dtype='str'),
         Index(['sex', 'bmi', 'bp', 's1', 's2', 's5'], dtype='str'),
         Index(['bmi', 'bp', 's1', 's2', 's5'], dtype='str'),
         Index(['bmi', 's1', 's2', 's5'], dtype='str'),
         Index(['bmi', 's1', 's5'], dtype='str'),
         Index(['bmi', 's5'], dtype='str'),
         Index(['bmi'], dtype='str')])
```

```
[array([ 1.75375799, -11.51180908, 25.60712144, 16.82887167,
        -44.44885564, 24.64095356, 7.67697768, 13.1387839 ,
         35.16119521, 2.35136365]),
 array([-11.25895092, 25.6246239 , 17.14383931, -44.68084867,
        25.15151718, 7.88275712, 12.90392433, 35.55515069,
         2.57745414]),
 array([-11.12122171, 25.99968678, 17.60752674, -45.2282964 ,
        25.75981894, 8.09181287, 13.25442301, 36.38970494]),
 array([-11.19779782, 26.05265839, 17.53804177, -31.5246206 ,
        16.33090209, 8.83532991, 31.74993939]),
 array([-10.24166307, 26.30983576, 16.94865316, -40.6326926 ,
        28.1145792 , 38.35744096]),
 array([ 28.22547979, 14.84176395, -31.28209119, 19.4831426 ,
        34.80035482]),
 array([ 32.64263283, -28.29576333, 17.26325665, 37.40448865]),
 array([ 34.8248582 , -10.89594022, 32.48784147]),
 array([34.56159785, 26.85219723]),
 array([47.14112176])],
 [10, 9, 8, 7, 6, 5, 4, 3, 2, 1])
```

Visualization

1.4 Task 4 - Analyze Feature Importance

```
[28]: # Using provided code from example RFE code so that I don't have to bash my
      ↪ head further trying
      # to figure out to add this specific formatting to my existing code.

import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

class CustomRFE:
    def __init__(self, estimator, n_features_to_select):
        self.estimator = estimator
        self.n_features_to_select = n_features_to_select
        self.coefficient_history = []
        self.eliminated_features = []

    def fit(self, X, y, feature_names):
        X_current = X.copy()
        features_remaining = list(range(X.shape[1]))

        while len(features_remaining) > self.n_features_to_select:
            # Fit linear regression
```

```

        self.estimator.fit(X_current, y)

        # Record current coefficients
        current_coefs = np.zeros(len(feature_names))
        current_coefs[features_remaining] = self.estimator.coef_
        self.coefficient_history.append(current_coefs)

        # Get absolute coefficient values for remaining features
        coef_abs = np.abs(self.estimator.coef_)

        # Find the feature with smallest absolute coefficient
        min_coef_idx = np.argmin(coef_abs)
        eliminated_feature_idx = features_remaining[min_coef_idx]

        # Track eliminated feature
        self.eliminated_features.
        ↪append(feature_names[eliminated_feature_idx])

        # Remove the feature with smallest coefficient
        features_remaining.pop(min_coef_idx)
        X_current = X[:, features_remaining]

    return self

```

```

[30]: max_num_features = 3
print(f'--- RFE with {max_num_features} Features ---')
rsqr_score, sel_feats, sel_coefs = perform_rfe_iteration(max_num_features);

print(f'Eliminated Features:')
for feat in X.columns:
    if feat not in sel_feats:
        print(feat)
print(f'\nSelected Features and Coefficients:')
for feat, coef in zip(sel_feats, sel_coefs):
    print(f'{feat}: {coef:.4f}')
print(f'\nR-Squared: {rsqr_score:.4f}\n')

# Fetch and prepare data
diabetes_2 = load_diabetes()
X = diabetes_2.data
feature_names = diabetes_2.feature_names
y = diabetes_2.target

# Standardize features

```

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Perform custom RFE
lr = LinearRegression()
custom_rfe = CustomRFE(lr, n_features_to_select=3)
custom_rfe.fit(X_scaled, y, feature_names)

# Create coefficient history DataFrame
coef_df = pd.DataFrame(
    custom_rfe.coefficient_history,
    columns=feature_names
)

# Highlight eliminated features in each iteration
print("Coefficient Values Across RFE Iterations:")
print(coef_df.round(4))

print("\nEliminated Features Order:")
for i, feature in enumerate(custom_rfe.eliminated_features, 1):
    print(f"Iteration {i}: {feature}")

```

--- RFE with 3 Features ---

Eliminated Features:

age
sex
bp
s2
s3
s4
s6

Selected Features and Coefficients:

bmi: 34.8249
s1: -10.8959
s5: 32.4878

R-Squared: 0.4451

Coefficient Values Across RFE Iterations:

	age	sex	bmi	bp	s1	s2 \
0	-0.47610000	-11.40690000	24.72650000	15.42940000	-37.68000000	22.67620000
1	0.00000000	-11.45520000	24.72960000	15.33030000	-37.61870000	22.56350000
2	0.00000000	-11.20130000	25.21420000	15.91020000	-37.90970000	22.91730000
3	0.00000000	-11.25950000	25.16460000	15.87870000	-29.25610000	16.79830000
4	0.00000000	-10.77380000	25.20380000	15.56400000	-36.05110000	25.61760000
5	0.00000000	0.00000000	26.75960000	13.03630000	-25.96860000	16.22420000
6	0.00000000	0.00000000	30.38990000	0.00000000	-24.82590000	15.68580000

	s3	s4	s5	s6
0	4.80610000	8.42200000	35.73440000	3.21670000
1	4.74300000	8.44080000	35.65010000	3.14740000
2	5.06640000	8.98060000	36.46290000	0.00000000
3	0.00000000	6.43390000	33.54740000	0.00000000
4	0.00000000	0.00000000	38.25130000	0.00000000
5	0.00000000	0.00000000	34.72950000	0.00000000
6	0.00000000	0.00000000	37.82170000	0.00000000

Eliminated Features Order:

Iteration 1: age

Iteration 2: s6

Iteration 3: s3

Iteration 4: s4

Iteration 5: sex

Iteration 6: bp

Iteration 7: s2

1. "Create a table showing the coefficients of each feature at each iteration of RFE." See above table.
2. "Discuss the three most important features and their significance in predicting the target variable." The most important features highlighted are bmi, s1, and s5. BMI seems to have the most predictive power, correlating strongly with having diabetes. Feature s5 also correlates strongly with having diabetes. Feature s1 somewhat correlates with NOT having diabetes.
3. "Compare the initial feature ranking with the final set of selected features" We can see that the bmi in its R^2 measurements: from 24.726 to 34.8249.

while S1 started quite strong, and ended up much lower compared to its initial ranking: -37.6800 to -10.8959

And s5 was increasing in its R^2 measurement, but then dropped much lower by the time of the final elimination pass: 35.7344 to 32.4878

1.5 Task 5 - Reflection

1. "What did you learn about feature selection using RFE?" I learned how RFE is a very effective way of measuring the goodness of fit for a given model. Visualizing this process by showing each iteration back to back was quite helpful for this task. Using the square residuals also natural way of addressing the absolute value issues. Squaring also will help with differentiation between features.
2. "How does RFE compare to other feature selection methods like LASSO in terms of methodology and results?" It seems like it will inevitably be more computationally heavy, since you are effectively retraining a model per iteration. But that will also result in a more fine grain reduction process. Lasso by comparison will just be faster, and more simple. Effectively you're trading speed for specificity.

Also RFE looks like it can handle more types of models than simple Linear Regression ones, even though that's what we used here.

3. “What insights can you draw about the dataset from the selected features?” That bmi has a has a strong explanatory power in predicting diabetes. This is consistent with what is known about Type 2 Diagnosis. S1 and S5 are unkown, but hold explanatory power. Age was quickly eliminated, likely because what predictive power it holds are better explained by the other features.